

AI-based Damage Detection for Power Grid Equipment with SaaS-oriented Design

Proposal Defense

Yujie Feng

Supervisor: Jin Hou

Project Vision: A Scalable SaaS Inspection Platform

AI Platform Core

- SaaS backend
 - unified auth / RBAC
 - multi-tenant isolation
 - task queue / data lake
- AI model registry
 - Module A: Bird Nest Detection (PoC)
 - Module B: Oil Leak Detection (Planned)
 - Module C: Infrared High-Temperature Detection (Planned)

Multi-modal Inputs

- UAV RGB imagery
- Infrared imagery
- Sensor telemetry streams
- Historical inspection records

Enterprise Outputs

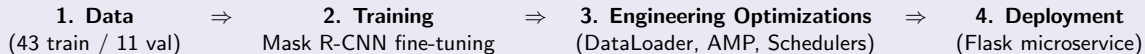
- Alert center & dashboard
- Maintenance workorders
- Visual analytics portal
- API & mobile integration

Accomplished (1): End-to-End PoC – Module A

Why start with bird-nest detection?

- High-frequency, high-risk issue with clear visual patterns; ideal to validate the pipeline.

Vertical slice implemented



COCO Validation Metrics

Metric	bbox	segm
AP@[0.50:0.95]	0.805	0.795
AP50	1.000	1.000
AP75	1.000	0.919
AR@[0.50:0.95]	0.837	0.828

Training Configuration

- Model: Mask R-CNN (ResNet50 + FPN v2)
- Dataset: 54 images (43 train, 11 val)
- Input size: 512×512 , batch size 4
- Optimizer: AdamW ($\text{lr} = 5 \cdot 10^{-4}$), OneCycleLR
- Epochs: 40 (early-stop ready)

PoC Progress (2): Training Dynamics

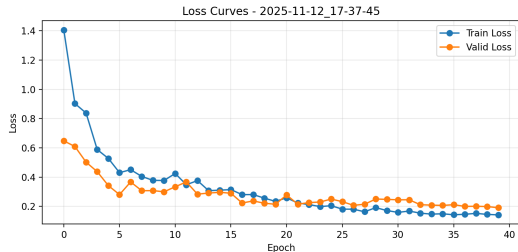


Figure: Train/Validation Loss

Key Observations

- Loss drops from 1.41 to 0.14 (train), 0.65 to 0.19 (val)
- Plateaus around epoch 20
- GPU utilization: 70–85% (after optimization)
- ~ About 27 seconds per epoch

PoC Progress (3): Precision-Recall Analysis

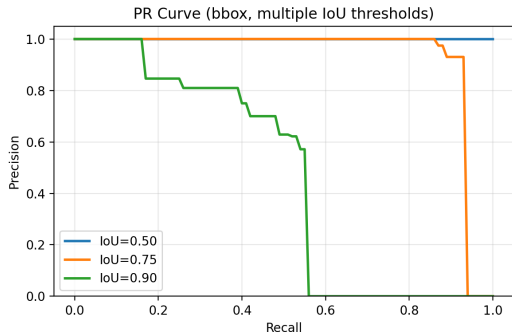


Figure: PR Curves (Bbox, IoU=0.50/0.75/0.90)

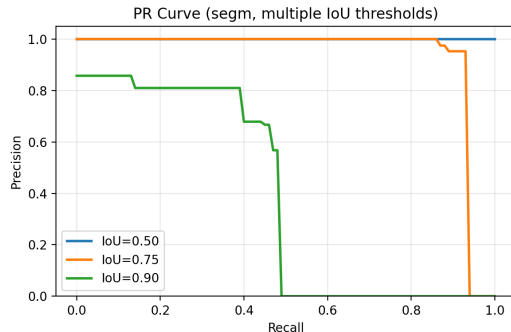


Figure: PR Curves (Segm, IoU=0.50/0.75/0.90)

IoU=0.50 stays close to precision 1.0; IoU=0.75 settles around 0.93, while IoU=0.90 dips toward 0.70, showing localization sensitivity.

PoC Progress (4): IoU Distributions & Threshold Analysis

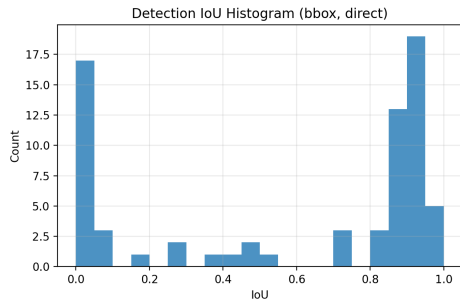


Figure: Bbox IoU Histogram

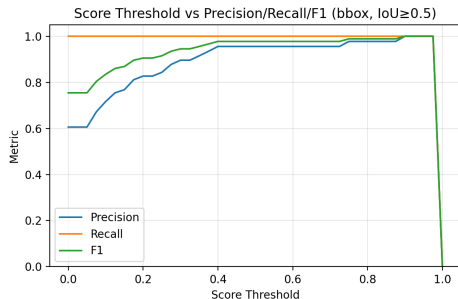


Figure: Score Threshold vs P/R/F1

Right-skewed IoU mass indicates strong localization. Operating threshold 0.4–0.5 balances recall and false positives.

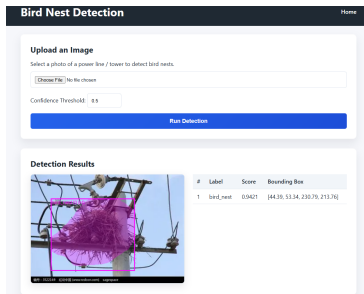
PoC Progress (5): Microservice-based Deployment

Flask REST API

- Accepts image upload
- Returns JSON (bbox, masks, scores)
- Returns annotated PNG
- Configurable confidence threshold

Significance

- Demonstrates “model as a service”
- First entry in AI model registry
- Enables integration with alert center
- Ready for multi-tenant SaaS backend



Key Insight: Generalization is the Core Challenge

The Trap of a "Perfect" Score

- $AP_{50} = 1.0$, $IoU \approx 1.0$ because validation data mirrors training scenes
- In real operations (fog, backlight, new structures) accuracy might drop sharply

Platform Lesson

- Any module (nest, oil leak, high-temperature) must solve generalization to new conditions
- Data augmentation, negative sampling, and multi-scene evaluation are platform-level requirements

Next Steps: Build Horizontally, Expand Vertically

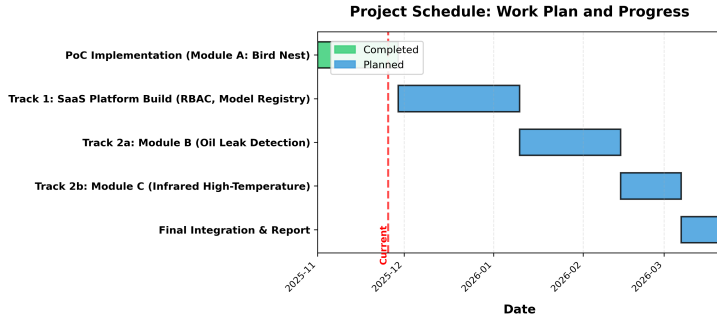
Track 1 – SaaS Platform (Weeks 4–10)

- Goal: Establish core infrastructure and standards
- Tasks:
 - 1 Implement multi-tenant backend with RBAC and task orchestration
 - 2 Define APIs for the AI model registry
 - 3 Productize PoC training utilities (augmentation, logging)

Track 2 – New Modules (Weeks 11–18)

- Goal: Validate extensibility
- Tasks:
 - 1 Kick off Module B (oil leak detection) or Module C (infrared high-temperature detection)
 - 2 Reuse platform capabilities for data processing and training
 - 3 Onboard new modules into the registry and microservice catalog

Work Plan and Schedule



Two-Track Approach:

- **Weeks 1–4 (Completed):** PoC Implementation (Module A)
- **Weeks 5–10:** Track 1 – SaaS Platform Build
- **Weeks 11–15:** Track 2a – Module B (Oil Leak)
- **Weeks 16–18:** Track 2b – Module C (Infrared)
- **Weeks 19–20:** Final Integration & Report

Discussion Points

- ① SaaS backend depth: should I prioritize feature completeness first (prototype RBAC) or invest in production-grade auth from day one?
- ② Module B technical choices: oil leak detection may require different approaches—should I consider semantic segmentation (e.g., U-Net) or stick with instance segmentation?
- ③ Module C integration: infrared high-temperature detection—should it be a standalone module or fused with RGB-based detection for multi-modal analysis?